

SELF-SUPERVISED LEARNING FOR ZERO-DAY UAV ATTACK DETECTION

Project 1 — Final Baseline Implementation and Evaluation Report

Platform	PX4 SITL + Gazebo X500
Data Source	PX4 ULog flight telemetry
Normal Flights	3
Normal Windows	476
Original Abnormal Windows	48
Window Size	20 samples
Window Step	5 samples
Telemetry Features	21
Transformer Embedding	128 dimensions
Controlled Attack	GPS WRONG failure injection

Purpose: establish and evaluate a self-supervised UAV telemetry anomaly-detection baseline using normal flight data and controlled PX4 SITL experiments.

1. Project Objective

The objective of Project 1 is to investigate whether a UAV telemetry representation learned from normal flight behavior can be used to identify previously unseen abnormal behavior. The implementation uses PX4 Software-In-The-Loop (SITL), Gazebo simulation, telemetry preprocessing, time-window construction, an autoencoder baseline, and a Transformer encoder trained using self-supervised contrastive learning.

The project is designed as a research baseline. The results demonstrate the behavior of the implemented methods on the collected simulation datasets; they should not be interpreted as proof of universal zero-day attack detection.

2. System Architecture

PX4 SITL → Gazebo X500 → MAVSDK / PX4 ULog → Telemetry Extraction → Preprocessing → Normalization → 20-sample Windows → Autoencoder Baseline + Transformer Contrastive Encoder → 128-D Embeddings → Anomaly Detection → Evaluation

Main telemetry features

Position: x, y, z; velocity: vx, vy, vz; acceleration: ax, ay, az; attitude: roll, pitch, yaw; angular velocity: angular_x, angular_y, angular_z; battery: voltage_v, remaining; actuator outputs: motor_1 through motor_4.

3. Experimental Environment

Component	Configuration
Operating system	Ubuntu 24.04.5 LTS in WSL2
Simulator	PX4 SITL
Vehicle	Gazebo X500
Gazebo	Gazebo Harmonic 8.15.0
Python	3.12.3
MAVSDK	3.17.2
NumPy	2.5.3
scikit-learn	1.9.1
TensorFlow	Installed in project virtual environment

4. Normal Flight Data Collection

Three normal SITL flights were generated using MAVSDK. Each flight performed takeoff, hover, directional movement, additional hovering, and landing. PX4 ULog files were extracted from the SITL log directory.

Flight	Processed Samples	Label
01	803	Normal
02	803	Normal
03	945	Normal
Total	2551	Normal

5. Data Preprocessing

PX4 ULog topics were extracted and aligned to a common 10 Hz timeline. Quaternion attitude data were converted to roll, pitch, and yaw. Telemetry streams were merged using nearest timestamps with a 0.15 s tolerance, followed by interpolation and forward/backward filling.

The constant `current_a` feature was removed during dataset combination. The final machine-learning input contains 21 telemetry features. Normalization used `StandardScaler` fitted only on normal data.

Window construction

Window size = 20 samples; step = 5 samples. At 10 Hz, each window represents approximately 2 seconds of telemetry and consecutive windows are shifted by approximately 0.5 seconds.

Dataset	Windows	Normal	Abnormal
Original evaluation dataset	524	476	48
Attack 01 dataset	1983	1729	254

6. Autoencoder Baseline

A reconstruction-based autoencoder was trained using only normal windows. The network flattens the 20×21 input and passes it through Dense layers of 128, 64, and 32 units before reconstructing the original 20×21 telemetry window.

Training used Adam with learning rate 0.001, mean squared error loss, 50 epochs, batch size 32, and a 20% validation split. The anomaly threshold was calculated from the normal reconstruction errors using mean + 3 standard deviations.

Metric	Result
Training windows	476 normal
Final training loss	0.041834
Final validation loss	0.205007
Threshold	0.536384
Accuracy	95.80%
Precision	68.57%
Recall	100.00%
F1	81.36%
TN / FP / FN / TP	454 / 22 / 0 / 48

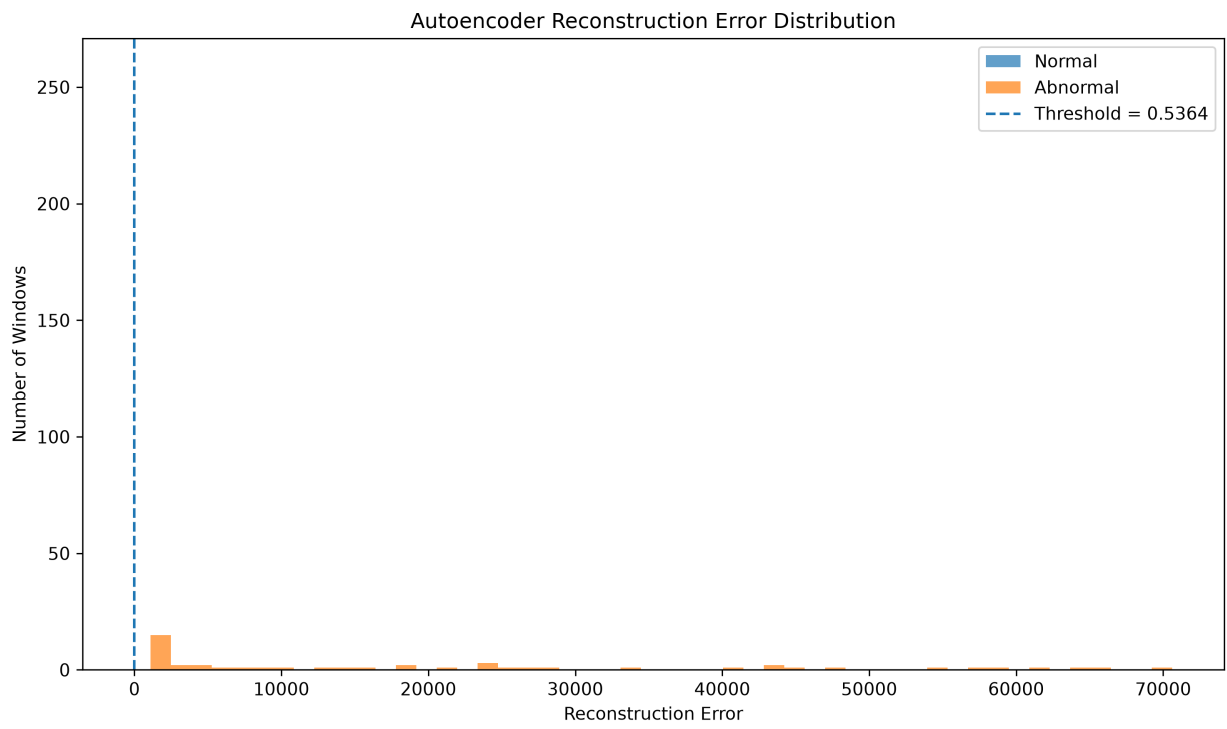


Figure 1. Autoencoder reconstruction-error distribution.

7. Self-Supervised Transformer

The Transformer receives a 20×21 telemetry window. A Dense projection maps the 21 features into a 128-dimensional representation and a learned positional embedding is added. Two Transformer blocks use 4-head self-attention and a 256-unit feed-forward network.

The final GlobalAveragePooling1D layer is followed by a 128-dimensional embedding layer. The resulting encoder contains 286,848 trainable parameters.

Contrastive learning

Two augmented views were generated for every normal window. The augmentations consisted of small Gaussian noise and small per-feature scaling. The encoder was trained with a symmetric NT-Xent / InfoNCE-style contrastive objective using temperature 0.1, Adam learning rate 0.0005, batch size 32, and 50 epochs.

The final contrastive loss was approximately 0.0459. The trained encoder was saved as `uav_transformer_encoder.keras`.

8. Transformer Anomaly Detection Methods

The learned 128-dimensional embeddings were evaluated using several distance or similarity-based approaches. The normal embeddings were used as the reference distribution.

Method	Description
Euclidean	Distance from normal embedding centroid.
Standard Mahalanobis	Covariance-aware distance using normal embeddings.
Robust Mahalanobis	MinCovDet-based robust covariance reference.
Cosine	1 – cosine similarity to normal embedding centroid.

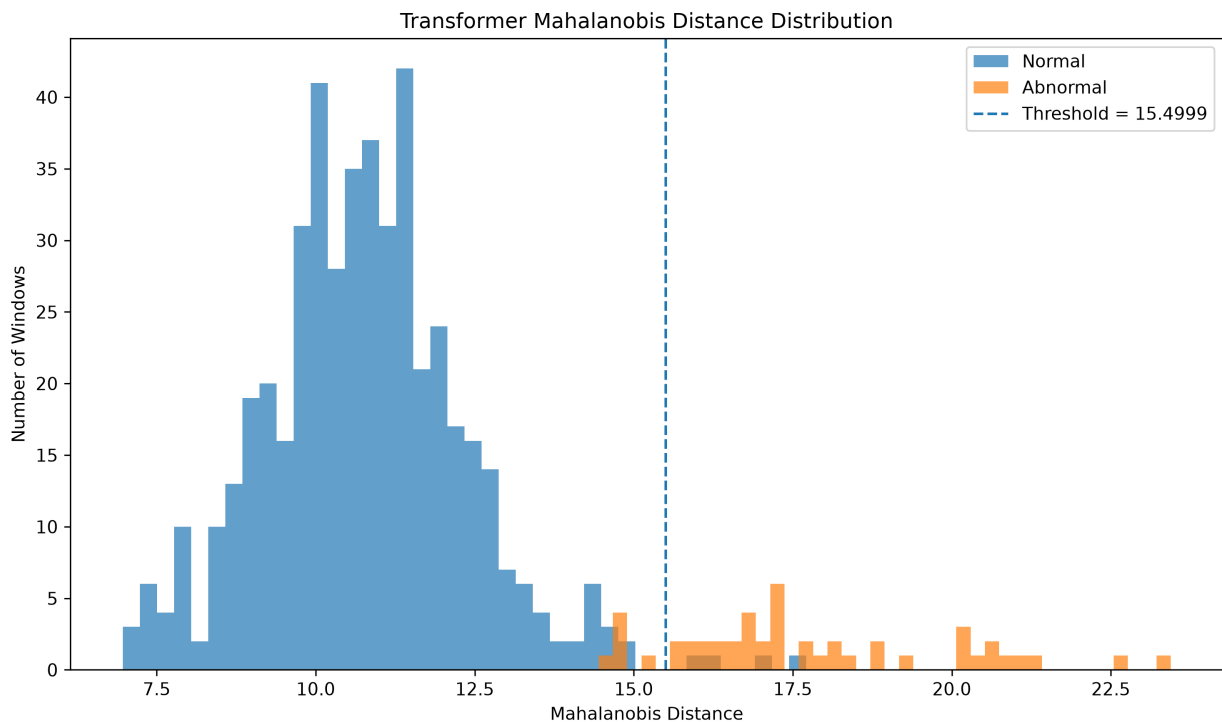


Figure 2. Standard Mahalanobis distance distribution.

9. Final Baseline Results

The following results were obtained on the 524-window original evaluation dataset containing 476 normal and 48 abnormal windows.

Method	Accuracy	Precision	Recall	F1
Autoencoder	95.80%	68.57%	100.00%	81.36%
Transformer + Euclidean	90.84%	0%	0%	0%
Transformer + Standard Mahalanobis	98.09%	91.30%	87.50%	89.36%
Transformer + Robust Mahalanobis	89.12%	0%	0%	0%
Transformer + Cosine	88.93%	36.84%	29.17%	32.56%

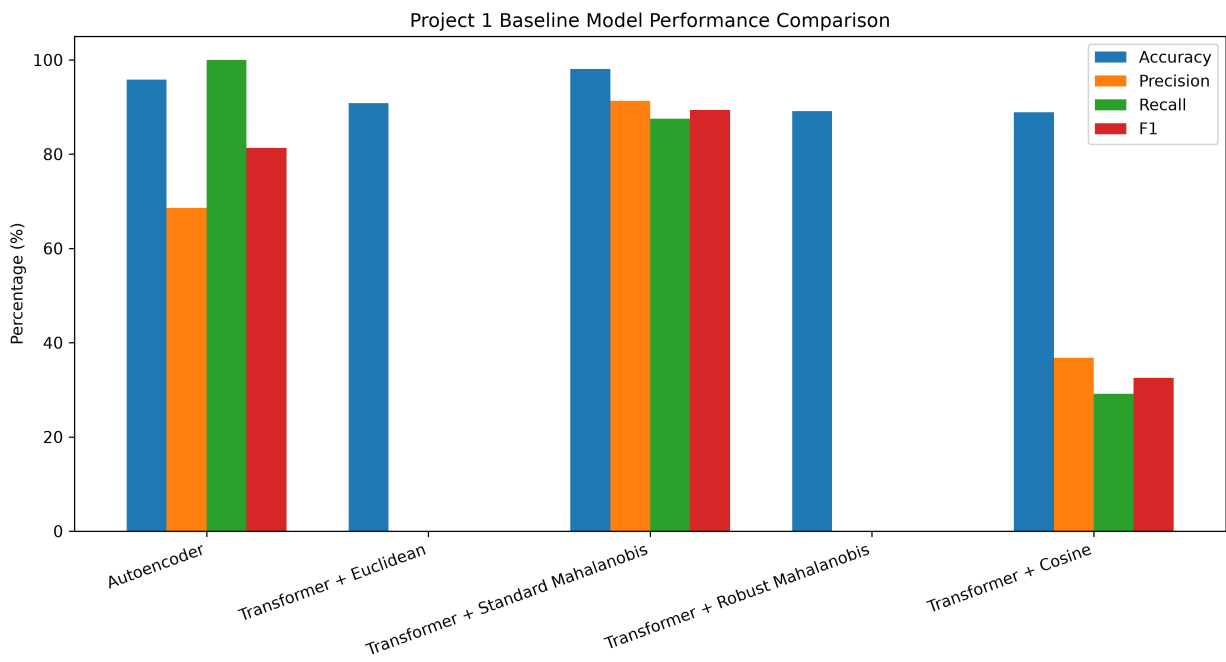


Figure 3. Experimental comparison of baseline detection methods.

10. Confusion Matrices

The standard Transformer + Mahalanobis detector produced 472 true normal classifications, 4 false positives, 6 false negatives, and 42 true abnormal classifications on the original evaluation dataset.

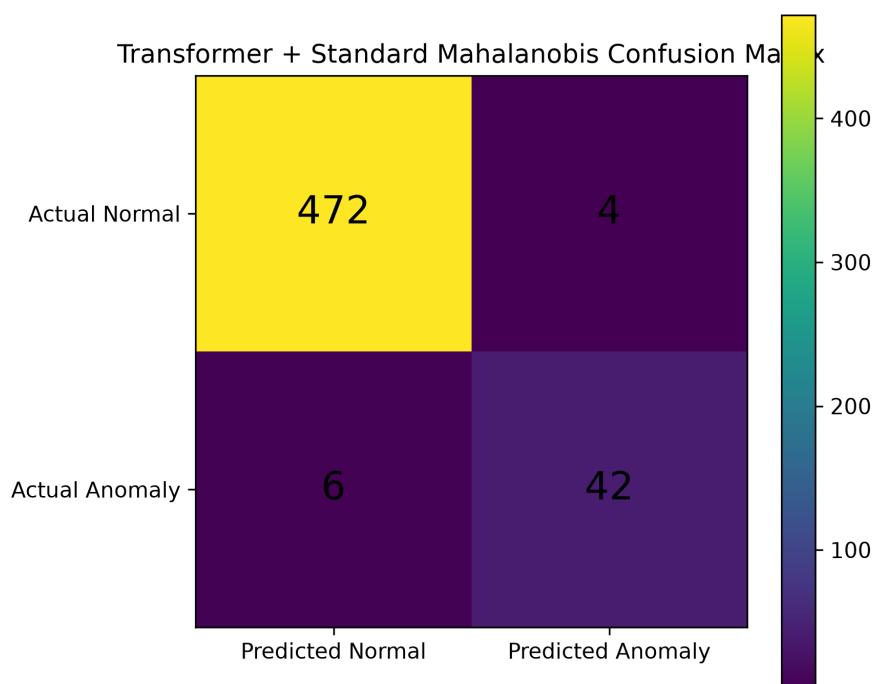


Figure 4. Transformer + Standard Mahalanobis confusion matrix.

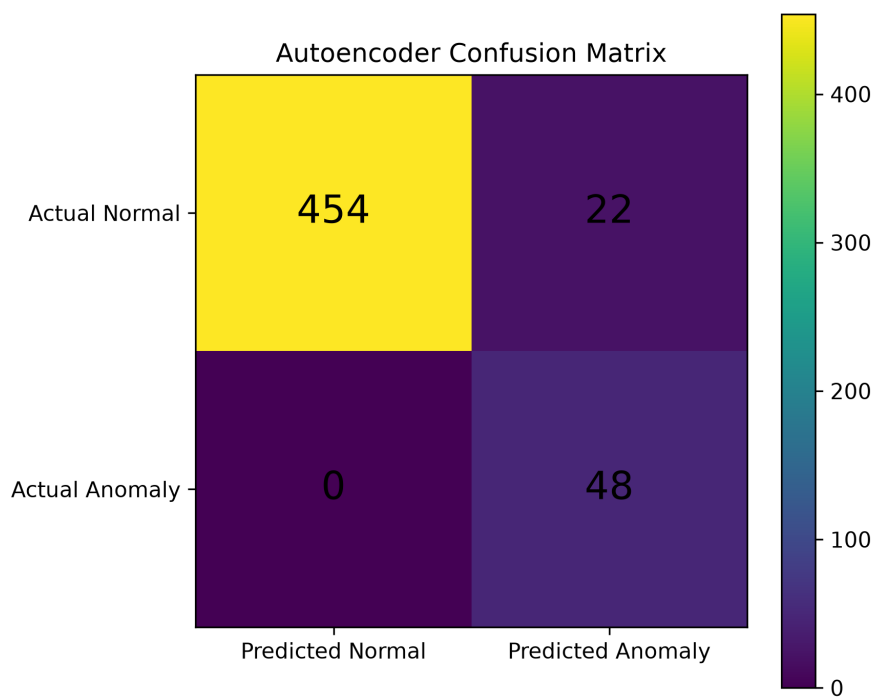


Figure 5. Autoencoder confusion matrix.

11. Controlled Attack 01 — GPS WRONG Failure Injection

A controlled PX4 SITL GPS failure-injection experiment was conducted using the MAVSDK failure API. PX4's GPS WRONG failure mode was enabled for approximately 10 seconds and then restored. This experiment is described as a controlled GPS anomaly/failure-injection scenario; it is not equivalent to a real-world GPS receiver spoofing attack.

The experiment generated a 79-second ULog. GPS-related topics contained 2,409 samples. The labeled attack interval was approximately 37.048 s to 47.048 s, corresponding to approximately 10 seconds of commanded failure injection.

Item	Result
Attack type	PX4 GPS WRONG failure injection
Attack duration	≈10.0 seconds
Attack raw telemetry rows	1,250
Attack windows	254
Attack dataset windows	1,983 total
Normal windows	1,729

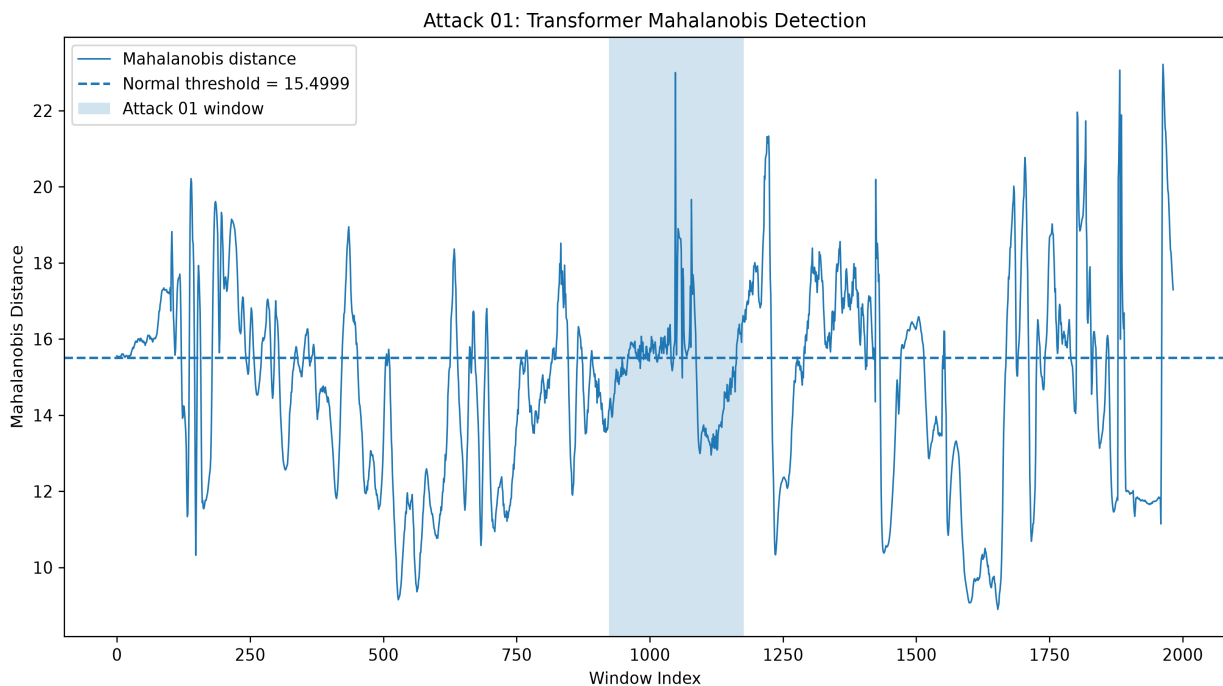


Figure 6. Transformer Mahalanobis distance during controlled Attack 01.

Attack 01 baseline evaluation

Metric	Result
Accuracy	54.92%
Precision	13.96%
Recall	48.82%
F1	21.72%
TN / FP / FN / TP	965 / 764 / 130 / 124

12. Experimental Findings

The autoencoder successfully reconstructed normal telemetry and produced very large reconstruction errors for the original high-speed abnormal event. Its recall on that evaluation set was 100%, although 22 normal windows were also classified as anomalous.

The self-supervised Transformer produced 128-dimensional telemetry representations. Among the tested distance methods, standard Mahalanobis distance provided the strongest measured result on the original 524-window evaluation set, with 98.09% accuracy and an F1 score of 89.36%.

The robust Mahalanobis and cosine experiments did not improve the measured baseline performance. Their results are retained as ablation experiments to document the behavior of alternative scoring methods.

Attack 01 produced a substantial performance reduction when the detector trained from the normal reference was evaluated on the separately generated controlled GPS failure-injection dataset. This demonstrates that performance on one abnormal behavior should not be interpreted as guaranteed zero-day generalization to every attack type.

13. Limitations

- The experiments were performed in PX4 SITL/Gazebo rather than on a physical UAV.
- The original abnormal dataset is a high-speed/abnormal movement event and is not a confirmed cyberattack.
- Attack 01 is a controlled PX4 GPS failure-injection experiment and not a real-world GPS spoofing measurement.
- The dataset is relatively small and contains only three normal flights for the baseline reference.
- The reported thresholds were derived from the available normal reference data.
- The current Transformer is an encoder trained with self-supervised contrastive learning; further work is required to establish broader zero-day generalization.

14. Future Work

- Generate multiple controlled UAV attack/failure scenarios in PX4 SITL.
- Evaluate whether attacks not represented during self-supervised training can be detected.
- Increase the number and diversity of normal flight trajectories.
- Investigate temporal contrastive augmentations and sequence-level objectives.
- Evaluate threshold calibration using a dedicated validation set.
- Compare against additional sequence anomaly-detection baselines.
- Test the trained encoder and detector on independent flight sessions.
- Investigate deployment on resource-constrained UAV/edge hardware.

15. Core Implementation Command Summary

```
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
cd ~/PX4-Autopilot
bash ./Tools/setup/ubuntu.sh
make px4_sitl gz_x500
cd ~/uav_project1
source .venv/bin/activate
python connect_test.py
python telemetry_test.py
python arm_test.py
python takeoff_test.py
python normal_flight.py
python preprocess_flight.py 01
python preprocess_flight.py 02
python preprocess_flight.py 03
python combine_normal_flights.py
python combine_labeled_dataset.py
python normalize_dataset.py
python create_windows.py
python train_autoencoder.py
python evaluate_autoencoder.py
python transformer_model.py
python create_contrastive_views.py
python train_transformer_contrastive.py
python test_gps_failure.py
python extract_attack01.py
python combine_attack01...
python evaluate_attack01...
```

Note: the command summary above records the principal implementation commands. The complete terminal history remains the authoritative record for exact one-line experimental commands used during development.

16. Key Project Files

- processed_data/normal_flights.csv
- processed_data/uav_labeled_dataset.csv
- processed_data/uav_normalized.csv
- processed_data/uav_windows_ml.npz
- processed_data/uav_autoencoder.keras
- processed_data/autoencoder_results.csv
- processed_data/uav_transformer_encoder.keras
- processed_data/all_transformer_embeddings.npz
- processed_data/transformer_mahalanobis_distances.npz
- processed_data/project1_baseline_results_summary.csv
- data_attack01/17_45_34.ulg
- data_attack01/attack01_labeled.csv
- data_attack01/attack01_windows_ml.npz
- data_attack01/attack01_baseline_results.csv
- data_attack01/attack01_transformer_embeddings.npz
- data_attack01/attack01_transformer_mahalanobis.npz

17. Conclusion

Project 1 establishes a complete experimental pipeline from PX4 SITL flight generation through telemetry preprocessing, self-supervised Transformer representation learning, anomaly scoring, and controlled attack evaluation. The strongest measured baseline on the original evaluation dataset was the Transformer with standard Mahalanobis distance. The separate GPS failure-injection experiment also revealed a significant generalization challenge, providing a concrete direction for the next stage of the research.

END OF PROJECT 1 BASELINE REPORT